

Week 12:

ROLL NO.:240801134

Name: Jenith Amala Sweety.A

Q1) A binary number is a combination of 1s and 0s. Its nth least significant digit is the nth digit starting from the right starting with 1. Given a decimal number, convert it to binary and determine the value of the the 4th least significant digit.

Example

number = 23

- Convert the decimal number 23 to binary number: $23_{10} = 2^4 + 2^2 + 2^1 + 2^0 = (10111)_2$.
- The value of the 4th index from the right in the binary representation is 0.

Function Description

Complete the function fourthBit in the editor below.

fourthBit has the following parameter(s):

int number: a decimal integer

Returns:

int: an integer 0 or 1 matching the 4th least significant digit in the binary representation of number.

Constraints

$0 \leq \text{number} < 2^{31}$

Input Format for Custom Testing

Input from stdin will be processed as follows and passed to the function.

The only line contains an integer, number.

Sample Input

STDIN Function

32 → number = 32

Sample Output

0

Explanation

- Convert the decimal number 32 to binary number: $32_{10} = (100000)_2$.
- The value of the 4th index from the right in the binary representation is 0.

Code:

```
int fourthBit(int number)
{
    int binary[32];
    int i=0;
    while(number>0)
    {
        binary[i]=number%2;
        number/=2;
        i++;
    }
    if(i>=4)
    {
        return binary[3];
    }
    return 0;
}
```

OUTPUT:

	Test	Expected	Got	
✓	printf("%d", fourthBit(32))	0	0	✓
✓	printf("%d", fourthBit(77))	1	1	✓

Passed all tests! ✓

Q2) Determine the factors of a number (i.e., all positive integer values that evenly divide into a number) and then return the *pth* element of the list, sorted ascending. If there is no *pth* element, return 0.

Example

$n = 20$

$p = 3$

The factors of 20 in ascending order are {1, 2, 4, 5, 10, 20}. Using 1-based indexing, if $p = 3$, then 4 is returned. If $p > 6$, 0 would be returned.

Function Description

Complete the function `pthFactor` in the editor below.

`pthFactor` has the following parameter(s):

`int n`: the integer whose factors are to be found

`int p`: the index of the factor to be returned

Returns:

`int`: the long integer value of the *pth* integer factor of *n* or, if there is no factor at that index, then 0 is returned

Constraints

$1 \leq n \leq 1015$

$$1 \leq p \leq 109$$

Input Format for Custom Testing

Input from stdin will be processed as follows and passed to the function.

The first line contains an integer n , the number to factor.

The second line contains an integer p , the 1-based index of the factor to return.

Sample Input

STDIN Function

10 $\rightarrow$ $n = 10$

3 $\rightarrow$ $p = 3$

Sample Output

5

Explanation

Factoring $n = 10$ results in $\{1, 2, 5, 10\}$. Return the $p = 3$ rd factor, 5, as the answer.

Code:

```
long pthFactor(long n, long p)
{
    int count=0;
    for(long i=1;i<=n;++i)
    {
        if(n%i==0)
        {
            count++;
            if(count==p)
            {
                return i;
            }
        }
    }
    return 0;
}
```

OUTPUT:

	Test	Expected	Got	
✓	<code>printf("%ld", pthFactor(10, 3))</code>	5	5	✓
✓	<code>printf("%ld", pthFactor(10, 5))</code>	0	0	✓
✓	<code>printf("%ld", pthFactor(1, 1))</code>	1	1	✓

Passed all tests! ✓

RECURSIVE FUNCTIONS

Status	Finished
Started	Sunday, 12 January 2025, 9:01 PM
Completed	Sunday, 12 January 2025, 9:07 PM
Duration	6 mins 18 secs

Q1) You are a bank account hacker. Initially you have 1 rupee in your account, and you want exactly N rupees in your account. You wrote two hacks, first hack can multiply the amount of money you own by 10, while the second can multiply it by 20. These hacks can be used any number of time. Can you achieve the desired amount N using these hacks.

Constraints:

$$1 \leq T \leq 100$$

$$1 \leq N \leq 10^{12}$$

Input

- The test case contains a single integer N.

Output

For each test case, print a single line containing the string "1" if you can make exactly N rupees or "0" otherwise.

SAMPLE INPUT

1

SAMPLE OUTPUT

1

SAMPLE INPUT

2

SAMPLE OUTPUT

0

Code:

```
int myFunc(int n)
{
    if(n==1)
    {
        return 1;
    }
    if(n%10==0)
    {
        if(myFunc(n/10))
        {
            return 1;
        }
    }
    if(n%20==0)
    {
        if(myFunc(n/20))
        {
            return 1;
        }
    }
    return 0;
}
```

OUTPUT:

	Test	Expected	Got	
✓	<code>printf("%d", myFunc(1))</code>	1	1	✓
✓	<code>printf("%d", myFunc(2))</code>	0	0	✓
✓	<code>printf("%d", myFunc(10))</code>	1	1	✓
✓	<code>printf("%d", myFunc(25))</code>	0	0	✓
✓	<code>printf("%d", myFunc(200))</code>	1	1	✓

Passed all tests! ✓

Q2)Q2) Find the number of ways that a given integer, X , can be expressed as the sum of the N th powers of unique, natural numbers.

For example, if $X = 13$ and $N = 2$, we have to find all combinations of unique squares adding up to 13. The only solution is $2^2 + 3^2$.

Function Description

Complete the `powerSum` function in the editor below. It should return an integer that represents the number of possible combinations.

`powerSum` has the following parameter(s):

X : the integer to sum to

N : the integer power to raise numbers to

Input Format

The first line contains an integer X .

The second line contains an integer N .

Constraints

$$1 \leq X \leq 1000$$

$$2 \leq N \leq 10$$

Output Format

Output a single integer, the number of possible combinations calculated.

Sample Input 0

10

2

Sample Output 0

1

Explanation 0

If $X = 10$ and $N = 2$, we need to find the number of ways that 10 can be represented as the sum of squares of unique numbers.

$$10 = 1^2 + 3^2$$

This is the only way in which 10 can be expressed as the sum of unique squares.

Sample Input 1

100

2

Sample Output 1

3

Explanation 1

$$100 = 10^2 = (6^2 + 8^2) = (1^2 + 3^2 + 4^2 + 5^2 + 7^2)$$

Sample Input 2

100

3

Sample Output 2

1

Explanation 2

100 can be expressed as the sum of the cubes of 1, 2, 3, 4.

(1 + 8 + 27 + 64 = 100). There is no other way to express 100 as the sum of cubes.

Code:

```
int powerSum(int x, int m, int n)
{
    if(x==0)
    {
        return 1;
    }
    if(x<0)
    {
        return 0;
    }
    int count=0;
    for(int i=m; ;i++)
    {
        int power=1;
        for(int j=0;j<n;j++)
        {
            power*=i;
        }
        if(power>x)
        {
            break;
        }
        count+=powerSum(x-power,i+1,n);
    }
    return count;
}
```

OUTPUT:

	Test	Expected	Got	
✓	printf("%d", powerSum(10, 1, 2))	1	1	✓

Passed all tests! ✓